

Incremento de Polimorfismo & Ligação Dinâmica

Sistema de Gerenciamento de Biblioteca – Projeto Integrado Java

Resumo do Incremento

Esta etapa do projeto introduz a abstração de itens de acervo por meio de um tipo genérico no topo da hierarquia de materiais. A solução elimina dependências diretas de tipos concretos no código cliente, garantindo flexibilidade para novos tipos de mídia e total aderência aos princípios SOLID.

ItemAcervo

Contrato Abstrato (Interface)

2 Subtipos

Livro & Revista (Concretos)

0 instanceof

Código Cliente Limpo

1. Contrato da Interface Abstrata (`ItemAcervo`)

A interface `ItemAcervo` foi projetada para estabelecer o contrato que qualquer material da biblioteca (livro, revista, mídias digitais) precisa obrigatoriamente cumprir. Toda documentação utiliza padrões Javadoc com precondições e exceções especificadas.

CÓDIGO FONTE — ITEMACERVO.JAVA (CONTRATO DOCUMENTADO)

```
package biblioteca; /** * Contrato que define os comportamentos essenciais de qualquer item
pertencente * ao acervo da biblioteca. */ public interface ItemAcervo { /** * Realiza o
empréstimo do item. * @throws IllegalStateException se o item não puder ser emprestado. */
void emprestar(); /** * Realiza a devolução do item ao acervo. * @throws
IllegalStateException se o item já estiver disponível. */ void devolver(); /** * Solicita a
reserva do item conforme as regras da mídia. * @throws IllegalStateException se estiver em
estado incompatível. */ void reservar(); /** * Obtém o título formatado e identificável do
item. * @return Nome/Título descritivo. */ String getTitulo(); }
```

2. Implementações Concretas Distintas (`Livro` e `Revista`)

Foram implementadas duas classes concretas para representar itens do acervo. Cada uma possui regras de negócio completamente justificadas pelo seu domínio de aplicação:

CRITÉRIO	CLASSE `LIVRO`	CLASSE `REVISTA`
Comportamento de Empréstimo	Altera o status para EMPRESTADO e decrementa a quantidade no estoque da classe Disponibilidade.	Altera a flag booleana de controle e aplica prazo reduzido de empréstimo.
Regra de Reserva	Permite reserva prévia desde que o livro não esteja emprestado no momento.	Bloqueia reserva prévia. Exibe mensagem explicativa (periódicos são para consulta no local).
Gestão de Estoque	Gerenciada por Composição de Objetos (relações Todo-Parte com Disponibilidade).	Controle individual direto por número de edição e cópia física.
Justificativa no Domínio	Livros são acervos permanentes, reutilizáveis e demandam filas de reserva organizadas.	Revistas são edições periódicas de alta rotatividade que não podem ficar bloqueadas para consulta rápida.

3. Código Cliente Polimórfico (`Main.java`)

O trecho de código a seguir demonstra o processamento do acervo manipulando unicamente referências do tipo abstrato `ItemAcervo`, sem qualquer acoplamento a tipos específicos:

TRECHO DO CÓDIGO CLIENTE — MAIN.JAVA

```
List<ItemAcervo> acervo = new ArrayList<>(); acervo.add(new Livro("Dante Alighieri",  
"Inferno de Dante", 101, 3)); acervo.add(new Revista("National Geographic", 204));  
acervo.add(new Livro("Robert C. Martin", "Clean Code", 102, 1)); acervo.add(new  
Revista("Superinteressante", 450)); // Processamento 100% polimórfico - Sem instanceof ou  
checagens condicionais for (ItemAcervo item : acervo) { System.out.println("Item: " +  
item.getTitulo()); try { item.emprestar(); // A JVM resolve o método concreto via Ligação  
Dinâmica } catch (IllegalStateException e) { System.out.println("Aviso: " + e.getMessage());  
} }
```

4. Justificativa Teórica: Polimorfismo vs. Herança Comum

A arquitetura adotada nesta etapa estabelece uma relação baseada estritamente em **Polimorfismo de Interface e Ligação Dinâmica**, diferindo substancialmente de uma herança de código comum.

1. Ausência de Reuso de Código

A interface `ItemAcervo` não compartilha atributos nem métodos concretos com `super`. Ela define apenas o contrato que as classes obrigatoriamente satisfazem.

2. Ligação Dinâmica (Dynamic Binding)

Ao executar `item.emprestar()`, a decisão de qual método rodar ocorre em tempo de execução pela JVM com base na instância real alocada na memória.

3. Princípio Open/Closed (OCP)

Novos tipos de acervo (como `Ebook` ou `Audiobook`) podem ser adicionados criando novas implementações de `ItemAcervo` sem alterar uma linha do código cliente.

4. Substituição de Liskov (LSP)

Qualquer objeto que implemente `ItemAcervo` pode substituir a referência genérica de maneira transparente sem quebrar o comportamento do sistema.

5. Validação do Incremento no Sistema

O incremento foi integrado ao projeto e validado através do tratamento de exceções de negócio e da suíte de testes unitários do JUnit.

Checklist de Requisitos Obrigatórios Atendidos

- ✓ Interface com nome expressivo (`ItemAcervo`) e documentação formal via Javadoc.
- ✓ Duas implementações concretas (`Livro` e `Revista`) com regras de negócio distintas.
- ✓ Código cliente baseado na interface abstrata sem nenhum operador `instanceof`.
- ✓ Tratamento seguro de exceções (`IllegalStateException` e `IllegalArgumentException`).
- ✓ Total integração ao sistema existente para a apresentação da AP2.